Lesson 7 — What Each Notebook Shows

Notebook guide, lesson 7 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

13 November 2026 · three notebooks

Contents

Notebook 01 — `01_decision_trees_from_scratch` **1**
Decision trees: splitting from scratch, and the depth dial 1

Notebook 02 — `02_bagging_and_random_forests` **2**
Bagging and random forests: averaging away the instability 2

Notebook 03 — `03_gradient_boosting` **4**
Gradient boosting: correcting mistakes in sequence 4

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — `01_decision_trees_from_scratch`

Decision trees: splitting from scratch, and the depth dial

What it is for. A fourth family, and the first one whose model a person can read. The notebook implements the splitting rule, checks it against scikit-learn, turns the depth dial, and ends on the instability that the next notebook exists to cure.

The data. 1,200 loan applicants, **38.7% defaulted**, with four separate risky patches — not one class surrounding another as in lesson 6.

The concepts, in order.

1. **Look at it first.** No straight line separates the classes, but logistic regression still reaches **0.748 ± 0.030** against a baseline of 0.613, because two of the four risky regions, very low income and very high debt, are monotonic in one feature. Not lesson 6's flat failure — a different shape of difficulty.
2. **A tree, split by split**: find the (feature, threshold) pair that makes the two resulting groups as pure as possible, apply it, repeat. Checked against scikit-learn at four depths and agreeing **exactly** — 0.8400, 0.8408, 0.8758, 0.9508. Then the root split in numbers: `debt_ratio <=`

`0.82` sends 971 left (p 0.256, G 0.3814) and 229 right (p 0.939, G 0.1148); weighted 0.3305 against the parent's 0.4743, **gain 0.1438** — drawn on the Gini curve (`gini_root_split.png`).

3. **Depth is the bias-variance dial**, and the table shows both ends:

depth	train	cross-validated
2	0.840	0.836
5	0.876	0.845
8	0.951	**0.882**
12	0.986	0.862
unconstrained	**1.000**	0.852

   Training accuracy climbs monotonically to 1.000; cross-validated accuracy peaks at depth 8 and falls back **while the model is only getting more capable**.

4. **What the tree can and cannot see.** At depth 2 it takes the income floor and the debt ceiling and misses both stressed islands; at depth 8 both islands appear as clean rectangles.

5. **The model a loan officer could apply by hand** — the printed if/else tree, nine leaves, readable top to bottom. This is what k-NN and the support vector machine (SVM) cannot offer.

6. **And the instability that ends the notebook.** Two resamples give root splits of **0.820 and 0.819** — the ceiling is overwhelming evidence — but the trees have 31 and 33 leaves and **agree on only 0.883 of predictions**. Stable at the top, unstable everywhere below.

The two-minute version. *A tree asks one question at a time and keeps asking until the group in front of it is nearly pure. We implement the Gini split from scratch and match scikit-learn prediction by prediction (100.0% identical) at every depth. Then depth is the dial: training accuracy climbs to a perfect 1.000 while cross-validated accuracy peaks at depth 8 and falls back. At depth 2 the tree finds the two big regions and misses the two islands; at depth 8 it finds all four. The last section is the reason notebook 2 exists: resample the same 1,200 applicants and the root split barely moves, but the two trees disagree on 12% of their predictions.*

Notebook 02 — `02_bagging_and_random_forests`

Bagging and random forests: averaging away the instability

What it is for. Notebook 1's instability looked like a weakness. Averaged over many trees, it becomes the mechanism.

The concepts, in order.

1. **Bagging from scratch**: bootstrap sample, fit a tree, repeat, vote. Test accuracy **0.872**, and scikit-learn's `BaggingClassifier` **0.872** — the same algorithm, not tree-for-tree identical, because the random streams differ.

2. **Why it works, and the out-of-bag consequence.** The chance a specific row is never drawn in m draws tends to $1/e$: **theory 0.3679**, $(1-1/m)^m$ at $m = 1200$ **0.3677**, simulated

0.3658. Those left-out rows give a free validation score: **OOB 0.9117 against cross-validated 0.9117 ± 0.0210** on this seed. The bootstrap counted: 0.366 of rows drawn never, 0.382 once, 0.168 twice, against the limit $e^{-1}/k!$ (`bootstrap_counts.png`).

3. **With the honesty the course requires.** Over twelve seeds the typical gap between OOB and CV is **0.0026** and the worst **0.0075**, matching to four decimals in 2 of

   12. The agreement on one seed is luck; the closeness is the property.

4. **Variance reduction, measured over 30 splits:**

	mean accuracy	std across splits
single tree	0.856	0.0185
bagging, 100 trees	0.902	0.0153
random forest, 100 trees	**0.908**	**0.0137**

   Before it, the floor drawn: $1/B + (B-1)/B \cdot \rho$ for four values of ρ — at $\rho = 0.5$, a hundred trees keep 0.505 of one tree's variance (`variance_floor.png`).

5. **What a random forest adds.** Bagging alone lets every tree see every feature, so a strong predictor gets picked first everywhere and the trees stay correlated. Restricting the candidate features per split decorrelates them.

6. **Feature importance, and its limits** — the section worth keeping:

noise columns	cv accuracy	importance on the noise
0	0.912	0.000
5	0.874	**0.339**
20	0.837	**0.545**

   With `max_features="sqrt"` and 22 columns, each split considers 4 of 22, so a specific real feature is a candidate with probability **0.182**, and **both real features are left out together with probability 0.662** — two splits in three offer only noise, and the tree must split on whichever noise column was drawn. **More than half the forest's importance lands on columns that contain nothing.**

The two-minute version. *One tree is unstable, so fit a hundred to bootstrap resamples and average them. The from-scratch version matches scikit-learn. Each bootstrap leaves out about 37% of the rows — 1/e, verified three ways — and those rows give a free validation score which lands within about 0.003 of cross-validation across seeds. Over 30 splits the variance falls: a single tree swings by ±0.018, a forest by ±0.014, and its mean accuracy is five points higher. The last section is a warning: add twenty noise columns and more than half of the forest's own feature importance lands on them, because with four candidates out of 22 per split, the real features are usually not even offered.*

Notebook 03 — `03_gradient_boosting`

Gradient boosting: correcting mistakes in sequence

What it is for. Bagging builds trees independently and averages them; boosting builds them **in sequence**, each one correcting what the ensemble so far got wrong.

The concepts, in order.

1. **Boosting from scratch, on regression**, where "what we got wrong" has an exact meaning — the residual. Mean squared error against the true function: **0.3947 after 1 tree, 0.0681 after 5, 0.0119 after 20, and 0.0178 after 60.** It gets worse again: the ensemble has started to track the noise.
2. **The same idea for classification**, boosting against the gradient of the log loss. It starts at the base rate's log-odds, **F0 = −0.461**, and the first tree is fit to residuals of **+0.613** (defaulters) and **−0.387** (repayers). Default `GradientBoostingClassifier`: **0.902 ± 0.020**.
3. **More trees always helps... until it does not.** Peak at **30 trees, cv 0.897**; training accuracy reaches **1.000 by about 200 trees** and stays; cv drifts down to 0.890 at 800. What a single unconstrained tree does in one step, boosting arrives at gradually.
4. **Learning rate, the same trade-off from the other side.** At 120 trees: $\alpha = 0.02$ gives cv 0.872, α **= 0.10 gives 0.902**, $\alpha = 1.00$ gives 0.878 with a perfect training score. α and `n_estimators` both control the total correction applied, which is why they trade against each other.
5. **The full leaderboard**, against a noise ceiling of 0.93:

model	cv accuracy	share of the ceiling
random forest, 100 trees	**0.911**	97.9%
bagging, 100 trees	0.904	97.2%
gradient boosting, 30 trees	0.897	96.5%
single tree, depth 8	0.882	94.9%
single tree, unconstrained	0.852	—
logistic regression	0.748	—
majority baseline	0.613	—

   All three ensembles land within two points of each other and closer to the ceiling than either single tree.

The two-minute version. *Boosting fits shallow trees in sequence, each one correcting the previous ensemble's mistakes. On regression you can watch it happen: one tree barely dents the flat guess, five sketch the shape, twenty trace it closely, and sixty start tracking the noise — the error goes back up. On our applicants, the peak is at 30 trees; push to 200 and the training accuracy is a perfect 1.000 while the honest score drifts down. The learning rate does the same thing from the other side, which is why the two parameters trade against each other. And the leaderboard: all three ensembles land within two points of one another and within about 3% of the noise ceiling, well above both single trees and far above logistic regression.*